

Álgebra Lineal para la Representación y Transformación de Datos

Guía completa para estudiantes de primer ingreso. No se presupone *ningún* conocimiento previo de álgebra lineal: empezamos en qué es un número ordenado en una lista y terminamos en el teorema espectral y las matrices dispersas que hacen posible el procesamiento del lenguaje natural.

PROF. RESPONSABLE · Departamento de Matemática Aplicada

NIVEL · Introductorio

ENFOQUE · Aplicaciones en IA y aprendizaje automático

CONTENIDO DEL CURSO

- | | |
|---|--|
| 1 · ¿Por qué el álgebra lineal es el idioma de la IA? | 8 · Rango, núcleo e imagen |
| 2 · Escalares, vectores, matrices y tensores | 9 · Determinantes y métodos de cálculo |
| 3 · Aritmética de vectores: suma, norma, producto punto | 10 · Matrices especiales y simétricas |
| 4 · Combinación lineal, span e independencia lineal | 11 · Autovalores, autovectores y teorema espectral |
| 5 · Espacios vectoriales, bases y dimensión | 12 · Matrices dispersas (sparse) |
| 6 · Transformaciones lineales | 13 · Todo junto en código (NumPy) |
| 14 · Mapa conceptual y glosario | 7 · Producto de matrices y composición |
| 15 · Ejercicios resueltos | |

1 · ¿Por qué el álgebra lineal es el idioma de la IA?

Una máquina no ve fotos, no lee palabras y no escucha música. Solo sabe hacer operaciones con **números organizados en listas y tablas**. Por eso, la primera tarea de cualquier sistema de inteligencia artificial es *traducir* el mundo a números; la segunda es *transformar* esos números hasta que la respuesta buscada aparezca. El álgebra lineal es exactamente la rama de las matemáticas que estudia esas dos tareas: **representar** datos y **transformarlos**.

IDEA CENTRAL DEL CURSO

Un **dato** es un punto en un espacio de muchas dimensiones. Un **modelo** es una serie de transformaciones de ese espacio. **Aprender** es buscar las transformaciones que colocan los puntos en posiciones donde la decisión se vuelve fácil.

Tres ejemplos de traducción a números

DATO DEL MUNDO REAL	REPRESENTACIÓN NUMÉRICA	TAMAÑO TÍPICO
Foto en escala de grises 28×28	Lista de 784 intensidades de gris (0 a 255)	784 números
Un paciente en un hospital	Edad, peso, presión, glucosa, ...	10-200 números
La palabra «gato»	Un <i>embedding</i> : lista de números aprendidos que codifican significado	300-4096 números

Fíjese en algo importante: en los tres casos el resultado es **lo mismo**, una lista ordenada de números. Da igual si venía de una foto o de una palabra. Ese objeto se llama **vector**, y es el ladrillo con el que construiremos todo lo demás.

2 • Escalares, vectores, matrices y tensores

Los cuatro objetos siguientes forman una escalera: cada uno es una colección ordenada del anterior. Lo único que cambia es el número de *índices* que hacen falta para señalar un número concreto.

0 ÍNDICES

Escalar

7.3

Un solo número.
La tasa de aprendizaje de una red.

1 ÍNDICE

Vector

[2, 0, 5]

Un ejemplo, una observación, una palabra.

2 ÍNDICES

Matriz

[[2,0],[1,4]]

Una tabla de datos, los pesos de una capa.

3+ ÍNDICES

Tensor

32×3×224×224

Un lote de imágenes en color: lote, canal, alto, ancho.

Notación que usaremos siempre. Los escalares en minúscula cursiva (a, λ); los vectores en minúscula negrita ($\mathbf{v}, \mathbf{x}$); las matrices en mayúscula (A, W). El elemento de la fila i y columna j de A se escribe a_{ij} . Un vector de n números vive en el conjunto $\mathbb{R}^n$ («erre a la ene»), que se lee «el espacio de todas las listas de n números reales».

CONVENCIÓN EN APRENDIZAJE AUTOMÁTICO

Un conjunto de datos se guarda como una matriz X de tamaño $m \times n$: **una fila por ejemplo** (los m pacientes, fotos o clientes) y **una columna por característica** (las n mediciones). Casi todos los errores de programación de los estudiantes de primer año nacen de confundir estos dos ejes.

```
X.shape = (60000, 784) → 60 000 imágenes, cada una con 784 píxeles
```

3 • Aritmética de vectores

3.1 Las dos operaciones fundamentales

Solo hay dos cosas que se pueden hacer con vectores, y de ellas se deriva todo el álgebra lineal: **sumarlos** (elemento a elemento) y **multiplicarlos por un escalar** (estirarlos o encogerlos). Geométricamente, sumar es «caminar primero por una flecha y luego por la otra»; multiplicar por un escalar es alargar la flecha, y si el escalar es negativo, darle la vuelta.

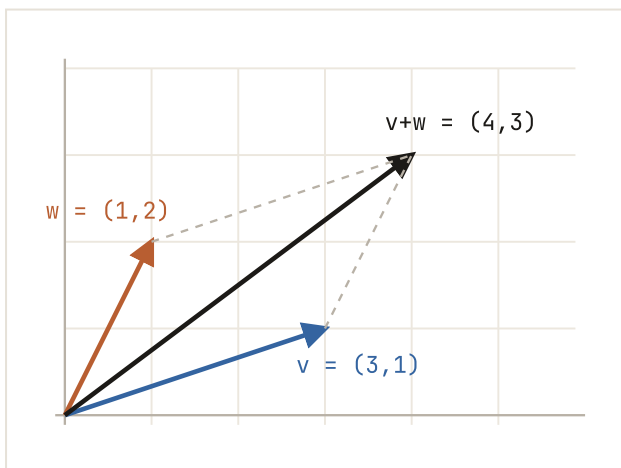


Fig. 1 – La suma de vectores es la diagonal del paralelogramo. Se suma coordenada por coordenada: $(3,1)+(1,2)=(4,3)$.

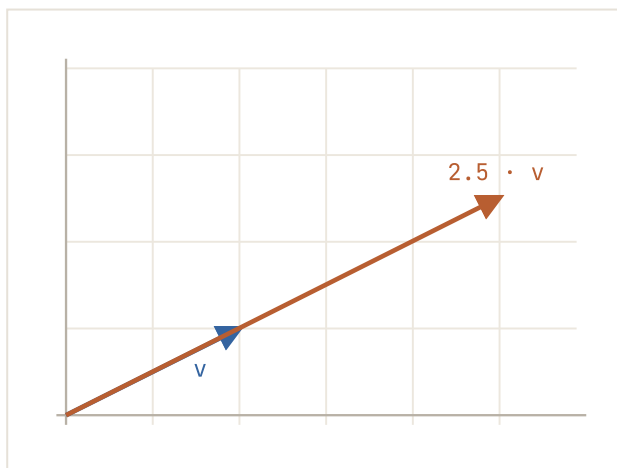


Fig. 2 – Multiplicar por un escalar cambia la longitud pero no la dirección (si el escalar es negativo, invierte el sentido).

3.2 Norma: ¿cuánto mide un vector?

$$\|v\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$$

Es el teorema de Pitágoras en n dimensiones: la longitud de la flecha. En IA la norma aparece constantemente: para medir el **error** de una predicción ($\|\hat{y} - y\|^2$), para **normalizar** vectores antes de compararlos, y para **regularizar** modelos penalizando pesos grandes (regularización L2 o *ridge*). La norma L1, $\|v\|_1 = \sum |v_i|$, suma valores absolutos y es la que produce modelos *dispersos*, con muchos pesos exactamente en cero (*lasso*).

3.3 Producto punto: la operación más importante de la IA

$$u \cdot v = u_1v_1 + u_2v_2 + \dots + u_nv_n = \|u\| \|v\| \cos \theta$$

El producto punto toma dos vectores y devuelve *un solo número* que mide cuánto «apuntan en la misma dirección». Si es positivo, forman un ángulo agudo; si es cero, son **perpendiculares** (ortogonales, sin nada en común); si es negativo, apuntan en direcciones opuestas.

Similitud del coseno

$\cos \theta = (u \cdot v) / (\|u\| \|v\|)$. Vale entre -1 y 1 y es la medida estándar para comparar *embeddings*: buscadores semánticos, sistemas de recomendación y RAG la usan millones de veces por segundo.

$\text{sim}(\text{«rey»}, \text{«reina»}) \approx 0.78$
 $\text{sim}(\text{«rey»}, \text{«tostadora»}) \approx 0.05$

Una neurona ES un producto punto

Una neurona artificial calcula $z = w \cdot x + b$ y luego aplica una función no lineal. Es decir: mide el parecido entre lo que recibe (x) y lo que ha aprendido a buscar (w).

$z = w \cdot x + b$
 $a = \sigma(z)$

3.4 Ejemplo numérico completo

$$u = (2, 3, 6) \quad v = (1, 0, -2)$$

$$u + v = (3, 3, 4)$$

$$3u = (6, 9, 18)$$

$$u \cdot v = 2 \cdot 1 + 3 \cdot 0 + 6 \cdot (-2) = 2 + 0 - 12 = -10$$

$$\|u\| = \sqrt{4 + 9 + 36} = \sqrt{49} = 7$$

$$\|v\| = \sqrt{1 + 0 + 4} = \sqrt{5} \approx 2.236$$

$$\cos \theta = -10 / (7 \cdot 2.236) \approx -0.639 \rightarrow \theta \approx 129.7^\circ \text{ (apuntan en sentidos casi opuestos)}$$

4 • Combinación lineal, span e independencia lineal

Una **combinación lineal** de los vectores $v_1, \dots, v_k$ es cualquier vector que se pueda escribir como $c_1 v_1 + c_2 v_2 + \dots + c_k v_k$ con escalares c_i . Es la única receta que tenemos: estirar y sumar. Al conjunto de *todos* los vectores que se pueden obtener así se le llama **span** (envolvente lineal o espacio generado).

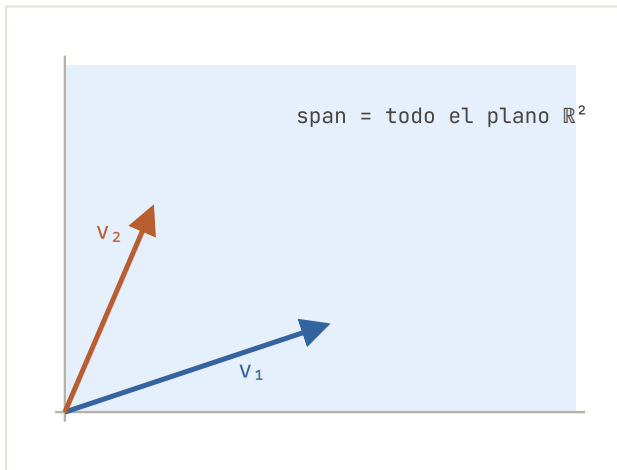


Fig. 3 – Vectores **independientes**: van en direcciones distintas, así que sus combinaciones cubren todo el plano (2 dimensiones de información).

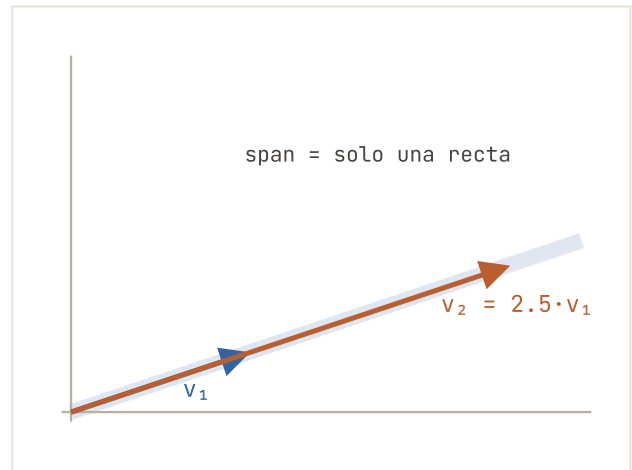


Fig. 4 – Vectores **dependientes**: uno es múltiplo del otro. Aunque hay dos vectores, solo aportan 1 dimensión de información.

DEFINICIÓN FORMAL • INDEPENDENCIA LINEAL

Los vectores $v_1, \dots, v_k$ son **linealmente independientes** si la única forma de obtener el vector cero como combinación suya es con todos los coeficientes nulos:

$$c_1 v_1 + \dots + c_k v_k = 0 \implies c_1 = c_2 = \dots = c_k = 0$$

Si existe alguna combinación con coeficientes no todos cero que dé cero, son **dependientes**: al menos uno es redundante, se puede escribir usando los demás y podría eliminarse sin perder información.

POR QUÉ LE IMPORTA A UN INGENIERO DE IA

Si en su tabla de datos incluye «altura en centímetros» y «altura en pulgadas», esas dos columnas son **linealmente dependientes**. El modelo no gana nada y aparecen problemas reales: la regresión lineal se vuelve inestable (multicolinealidad), los coeficientes explotan y dejan de ser interpretables. Detectar dependencia lineal es, en la práctica, detectar *información duplicada*. Ese es también el fundamento de la reducción de dimensionalidad: quedarse con un conjunto pequeño de direcciones independientes.

5 • Espacios vectoriales, bases y dimensión

Un **espacio vectorial** es un conjunto de objetos que se pueden sumar y multiplicar por escalares sin salirse del conjunto, cumpliendo unas reglas de sentido común. La definición parece abstracta, pero su utilidad es enorme: cualquier cosa que satisfaga esas reglas hereda *todo* el álgebra lineal. Y muchas cosas las satisfacen: las listas de números, las matrices, los polinomios, las funciones, las señales de audio, e incluso el espacio de los *embeddings* de un modelo de lenguaje.

AXIOMA	EN PALABRAS LLANAS
$u+v \in V$	Al sumar dos elementos sigo dentro del conjunto (cerrado bajo la suma).
$c \cdot v \in V$	Al escalar un elemento sigo dentro (cerrado bajo el producto por escalar).
$u+v = v+u$	El orden de la suma no importa (conmutativa) y agrupar da igual (asociativa).
$\exists \ 0$	Existe el vector cero, que sumado no cambia nada.
$\exists \ -v$	Todo elemento tiene opuesto: $v + (-v) = 0$.
$c(u+v) = cu+cv$	Escalar y sumar se distribuyen; $1 \cdot v = v$.

5.1 Subespacio

Un **subespacio** es un espacio vectorial que vive dentro de otro: una recta o un plano que *pasan por el origen*. La condición «por el origen» no es un capricho: si no contiene el 0, no es cerrado bajo el producto por escalar. Los subespacios de $\mathbb{R}^3$ son exactamente cuatro tipos: el origen solo (dimensión 0), cualquier recta por el origen (1), cualquier plano por el origen (2) y todo $\mathbb{R}^3$ (3).

5.2 Base y dimensión

Una **base** es un conjunto de vectores que (a) son linealmente independientes y (b) generan todo el espacio. Es el «sistema de coordenadas mínimo»: con ella, cada vector se escribe de *una única* manera. El número de vectores de la base es la **dimensión** del espacio, y no depende de la base elegida. La base más simple de $\mathbb{R}^3$ es la **canónica**: $e_1=(1,0,0)$, $e_2=(0,1,0)$, $e_3=(0,0,1)$.

CAMBIO DE BASE = EL CORAZÓN DEL APRENDIZAJE DE REPRESENTACIONES

Los mismos datos descritos en otra base pueden ser mucho más fáciles de manejar. PCA busca una base en la que las direcciones estén ordenadas por cantidad de información; la transformada de Fourier es un cambio a la base de frecuencias (la que usan MP3 y JPEG); y las capas ocultas de una red neuronal profunda no son otra cosa que **bases aprendidas** en las que el problema se vuelve linealmente separable. Cuando alguien dice «espacio latente», está diciendo «una base nueva y más útil».

6 • Transformaciones lineales

Una **transformación lineal** T es una función que toma vectores y devuelve vectores respetando las dos operaciones fundamentales:

$$T(u + v) = T(u) + T(v) \quad T(c \cdot v) = c \cdot T(v)$$

Intuitivamente: las rectas siguen siendo rectas, el origen se queda quieto y las cuadrículas paralelas se mantienen paralelas y equiespaciadas. Nada de curvas, nada de desplazamientos. **Teorema fundamental:** toda transformación lineal de $\mathbb{R}^n$ a $\mathbb{R}^m$ se puede escribir como una multiplicación por una matriz A de tamaño $m \times n$, y las columnas de A son simplemente las imágenes de los vectores de la base canónica. Una matriz no es una tabla muerta de números: **es un verbo**.

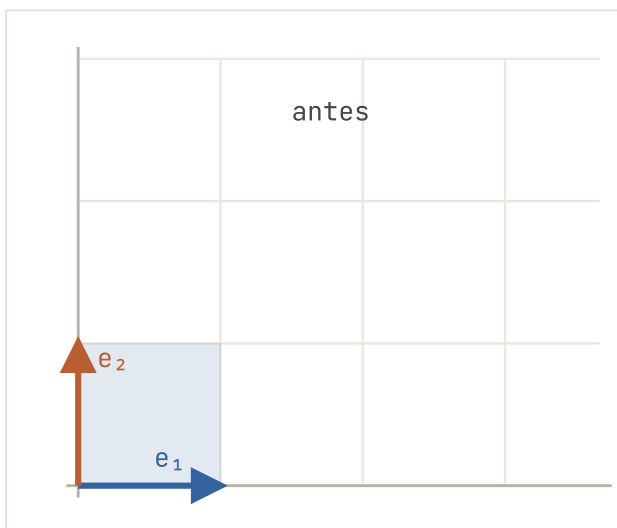


Fig. 5a – El cuadrado unidad y la base canónica.

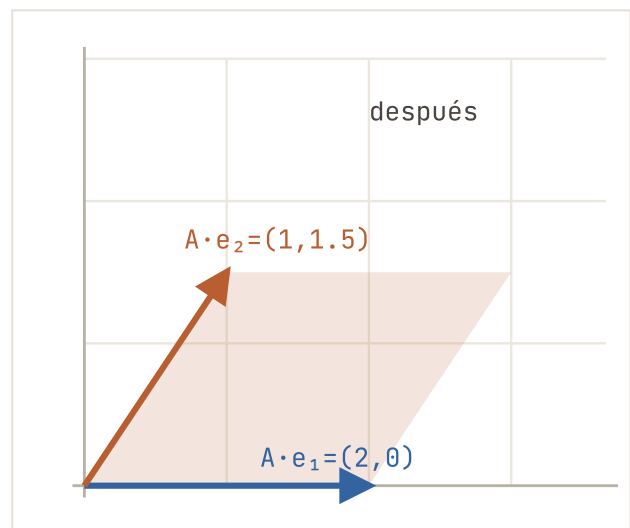


Fig. 5b – Tras aplicar A : el cuadrado se convierte en paralelogramo. Las columnas de A son las flechas nuevas.

TRANSFORMACIÓN	MATRIZ (2×2)	DÓNDE APARECE EN IA
Escalado	$\begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$	Normalización de características, aumento de datos
Rotación (θ)	$\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$	Visión por computador, codificación posicional rotatoria
Cizalla (shear)	$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$	Aumento de datos en imágenes
Proyección sobre el eje x	$\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$	Reducción de dimensión, capas de cuello de botella
Capa densa (sin sesgo)	W ($m \times n$ cualquiera)	Toda red neuronal, proyecciones Q/K/V en transformers

¿Y el sesgo b ? La operación de una capa neuronal, $Wx + b$, no es estrictamente lineal (mueve el origen): se llama *afín*. Y sobre esto conviene ser explícito desde el primer día: si una red solo apilara transformaciones lineales, el resultado global sería otra transformación lineal, y toda la profundidad no serviría de nada. Por eso entre capa y capa se coloca una función **no lineal** (ReLU, sigmoide, GELU). El álgebra lineal aporta la potencia de cálculo; la no linealidad aporta la expresividad.

7 • Producto de matrices y composición

El elemento (i,j) del producto AB es el **producto punto de la fila i de A por la columna j de B**. Para que la operación exista, el número de columnas de A debe coincidir con el de filas de B: $(m \times k) \cdot (k \times n) = (m \times n)$. Esa regla de las dimensiones es la fuente número uno de errores al programar modelos.

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$AB = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

$$BA = \begin{bmatrix} 23 & 34 \\ 31 & 46 \end{bmatrix} \rightarrow AB \neq BA : \text{el producto NO es conmutativo}$$

Que no sea conmutativo tiene un sentido geométrico claro: «rotar y luego estirar» no es lo mismo que «estirar y luego rotar». Multiplicar matrices **es componer transformaciones**, y el orden de las acciones importa. En cambio sí es asociativo, $A(BC)$

= (AB)C, y de ahí sale una optimización muy real: en una red profunda, agrupar bien los productos cambia el coste de cómputo en órdenes de magnitud.

LA OPERACIÓN QUE CONSUME EL 90 % DE LA ENERGÍA DE LA IA

Pasar un lote completo de datos por una capa es *una sola* multiplicación de matrices: $H = XW + b$, donde X es (lote $\times$ entradas) y W es (entradas $\times$ salidas). Por eso existen las GPU y las TPU: son máquinas especializadas en multiplicar matrices en paralelo. Entrenar un modelo es, literalmente, hacer esta operación billones de veces.

$X (64 \times 784) \cdot W (784 \times 128) = H (64 \times 128) \rightarrow \text{coste} \approx 6.4 \text{ millones de multiplicaciones}$

8 • Rango, núcleo e imagen

Estos tres conceptos responden a una única pregunta: *¿cuánta información conserva una transformación y cuánta destruye?*

Imagen (Im A)

Todo lo que la transformación *puede* producir: el span de las columnas de A (también llamado *espacio columna*). Es un subespacio del espacio de llegada.

Rango (rank A)

La *dimensión* de la imagen: cuántas columnas son realmente independientes. Mide la información útil que atraviesa la transformación.

Núcleo (Ker A)

Todos los vectores que la transformación aplasta al cero: $Ax = 0$. Es la información *destruida*, irrecuperable. Su dimensión es la *nulidad*.

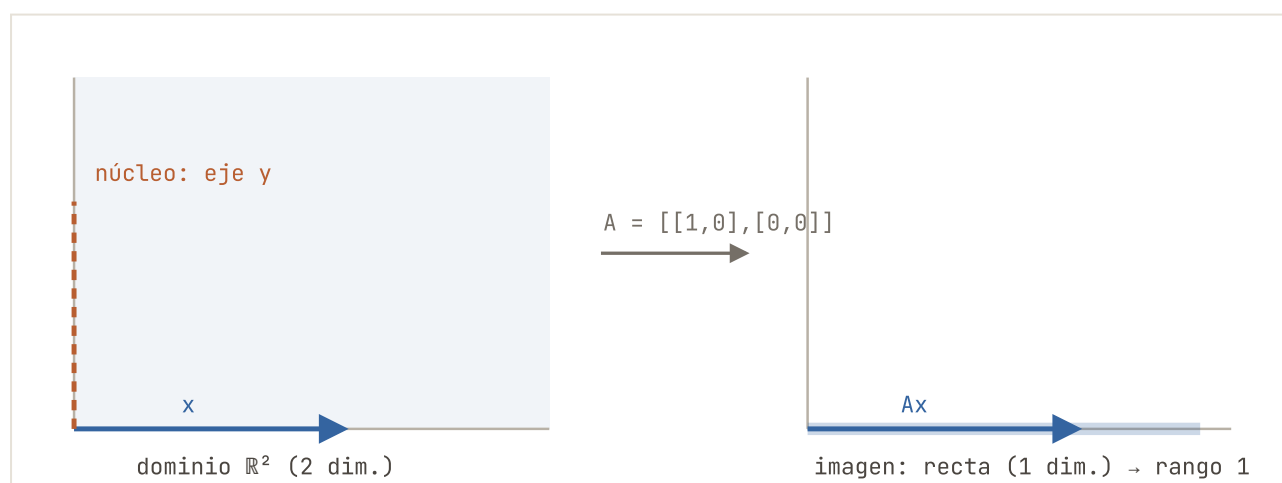


Fig. 6 – Una proyección aplasta el plano sobre una recta. Rango 1 (lo que sobrevive) + nulidad 1 (el eje y , que se pierde) = 2, la dimensión del dominio.

TEOREMA DEL RANGO Y LA NULIDAD

$$\text{rank}(A) + \dim(\text{Ker } A) = n \quad (n = \text{número de columnas})$$

Ley de conservación de la información: las dimensiones de entrada se reparten entre las que sobreviven y las que se destruyen. Si $\text{Ker } A = \{0\}$, nada se pierde y la transformación es **inyectiva** (invertible sobre su imagen). Una matriz cuadrada $n \times n$ con $\text{rank} = n$ se llama de **rango completo** y es invertible; si $\text{rank} < n$ es **singular**.

8.1 Cómo se calcula el rango en la práctica

Se aplica **eliminación de Gauss** hasta llegar a una forma escalonada y se cuentan las filas no nulas (los *pivotes*). Ejemplo:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 1 \end{bmatrix} \xrightarrow{\substack{F_2 \leftarrow F_2 - 2 \cdot F_1 \\ F_3 \leftarrow F_3 - F_1}} \begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 0 & -1 & -2 \end{bmatrix}$$

Filas no nulas = 2 $\rightarrow$ **rank(A) = 2**, nulidad = 3 - 2 = 1, A es singular.

La fila 2 era el doble de la fila 1: información duplicada.

RANGO BAJO: DE DEFECTO A HERRAMIENTA

Una matriz de datos de rango bajo significa que las columnas son redundantes; explotar eso es el fundamento de la **compresión** y de los **sistemas de recomendación** (factorización de matrices: la enorme matriz usuarios×películas se aproxima por dos matrices delgadas). También es la idea de **LoRA**, la técnica estándar para ajustar modelos de lenguaje gigantes: en lugar de reentrenar W , se le suma una corrección de rango bajo $\Delta W = BA$ con B y A muy delgadas, reduciendo los parámetros entrenables en más del 99 %. Y en un autoencoder, la capa de cuello de botella impone deliberadamente un rango pequeño para forzar al modelo a quedarse solo con lo esencial.

9 • Determinantes y métodos para obtenerlos

El **determinante** de una matriz cuadrada es un único número, $\det(A)$ o $|A|$, con un significado geométrico precioso: **el factor por el que la transformación multiplica el área (en 2D) o el volumen (en 3D)**. Si $\det(A) = 3$, las figuras triplican su área. Si $\det(A) = 0$, la transformación aplasta el espacio a algo de dimensión menor: se pierde información, no hay vuelta atrás y **la matriz no es invertible**. Un determinante negativo indica que además se invierte la orientación (como un espejo).

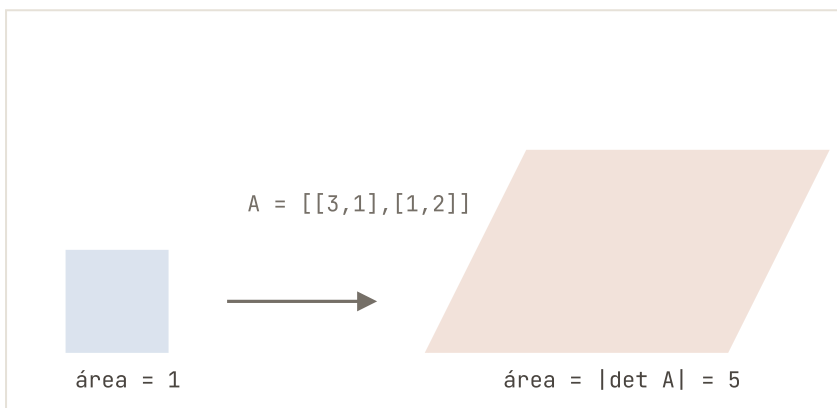


Fig. 7 – El determinante es el factor de cambio de área. El cuadrado unidad se convierte en un paralelogramo de área $|\det A|$.

Método 1 • Matriz 2×2 (fórmula directa)

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \rightarrow \det(A) = a \cdot d - b \cdot c$$

$$\text{Ejemplo: } \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix} \rightarrow 3 \cdot 2 - 1 \cdot 1 = 5$$

Método 2 • Regla de Sarrus (solo 3×3)

Se suman los tres productos de las diagonales que descienden hacia la derecha y se restan los tres de las que descienden hacia la izquierda. Es cómodo a mano, pero *no* se generaliza a tamaños mayores.

$$\det = aei + bfg + cdh - ceg - bdi - afh$$

$$\begin{aligned} A &= \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{bmatrix} \\ &= (2 \cdot 3 \cdot 2) + (1 \cdot 1 \cdot 0) + (0 \cdot 1 \cdot 1) - (0 \cdot 3 \cdot 0) - (1 \cdot 1 \cdot 2) - (2 \cdot 1 \cdot 1) \\ &= 12 + 0 + 0 - 0 - 2 - 2 = 8 \end{aligned}$$

Método 3 • Desarrollo por cofactores (Laplace) — cualquier tamaño

Se elige una fila o columna (conviene la que tenga más ceros) y se descompone en determinantes más pequeños. El **menor** M_{ij} es el determinante que queda al tachar la fila i y la columna j ; el **cofactor** es $C_{ij} = (-1)^{i+j} M_{ij}$, con el tablero de signos $+-+/-+-/+ -+$.

$$\det(A) = \sum_j a_{1j} \cdot C_{1j} \text{ (desarrollo por la primera fila)}$$

$$\begin{aligned} A &= \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{bmatrix} \text{ desarrollando por la 1ª fila:} \\ &= +2 \cdot \det \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix} - 1 \cdot \det \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix} + 0 \cdot \det \begin{bmatrix} 1 & 3 \\ 0 & 1 \end{bmatrix} \\ &= 2 \cdot (6-1) - 1 \cdot (2-0) + 0 = 10 - 2 = 8 \checkmark \text{ coincide con Sarrus} \end{aligned}$$

Coste: $O(n!)$ — para $n=20$ son más operaciones que átomos en una ciudad.
Inviabile en computación.

Método 4 • Triangulación por eliminación de Gauss (el que usa el ordenador)

El determinante de una matriz triangular es el producto de su diagonal. Así que se lleva A a forma triangular con operaciones elementales, controlando cómo afecta cada una:

OPERACIÓN ELEMENTAL	EFFECTO SOBRE EL DETERMINANTE
Sumar a una fila un múltiplo de otra	No lo cambia
Intercambiar dos filas	Cambia de signo
Multiplicar una fila por k	Lo multiplica por k

```
[[2, 1, 0], F2←F2-½F1 [[2, 1, 0], F3←F3-0.4F2 [[2, 1, 0],
[1, 3, 1], —————> [0, 2.5, 1], —————> [0, 2.5, 1],
[0, 1, 2]] [0, 1, 2]] [0, 0, 1.6]]
```

$\det = 2 \cdot 2.5 \cdot 1.6 = 8$ (ninguna operación alteró el valor)

Coste: $O(n^3)$. Es lo que hacen internamente `numpy.linalg.det` y la descomposición LU.

Propiedades y usos en IA

$$\det(AB) = \det(A) \cdot \det(B)$$

$$\det(A^T) = \det(A)$$

$$\det(A^{-1}) = 1/\det(A)$$

$$\det(cA) = c^n \cdot \det(A)$$

$\det(A)$ = producto de los autovalores

$\det(A) = 0 \iff A$ singular $\iff \text{rank} < n$

En aprendizaje automático el determinante aparece sobre todo en probabilidad: la densidad de una **gaussiana multivariante** incluye el término $\det(\Sigma)$ de la matriz de covarianza, y los modelos generativos de **flujos normalizantes** se basan en corregir la densidad con el determinante del jacobiano de cada transformación. En la práctica se calcula el *logaritmo* del determinante (`slogdet`) para evitar desbordamientos numéricos.

10 • Matrices especiales y matrices simétricas

TIPO	CONDICIÓN	SIGNIFICADO / USO
Identidad I	unos en la diagonal	No hace nada: $AI = A$. Es el «1» de las matrices. Base de las conexiones residuales.
Diagonal D	$a_{ij}=0$ si $i \neq j$	Escala cada eje por separado. Barata de invertir y de multiplicar.
Transpuesta A^T	filas $\leftrightarrow$ columnas	Omnipresente en retropropagación: el gradiente vuelve multiplicando por W^T .
Simétrica	$A = A^T$	Espejo respecto a la diagonal. Covarianzas, matrices de Gram, kernels, hessianos, grafos no dirigidos.
Ortogonal Q	$Q^T Q = I$	Rota o refleja sin deformar: conserva longitudes y ángulos. Inicialización estable de redes.
Definida positiva	$x^T A x > 0 \quad \forall x \neq 0$	Todos los autovalores positivos. Garantiza un mínimo único: optimización convexa.

Las **matrices simétricas** merecen atención especial porque son las que uno encuentra de verdad al analizar datos. La razón es estructural: cualquier matriz de datos X genera automáticamente dos matrices simétricas, $X^T X$ (relaciones entre características, la *covarianza* si los datos están centrados) y XX^T (relaciones entre ejemplos, la matriz de *Gram* o de *kernel*). «Cuánto varía A junto con B» es lo mismo que «cuánto varía B junto con A»: de ahí la simetría.

Matriz de covarianza de 3 características (simétrica, 3×3):

```
Σ = [[ 4.0, 2.1, -0.3],  
      [ 2.1, 3.5, 0.8],  
      [-0.3, 0.8, 1.2]]
```

diagonal → varianza de cada característica · fuera de la diagonal → covarianzas

$\Sigma[0][1] = \Sigma[1][0] = 2.1 \rightarrow$ las características 1 y 2 crecen juntas

11 • Autovalores, autovectores y teorema espectral

Cuando aplicamos una matriz, casi todos los vectores cambian de dirección. Pero suele haber unas pocas direcciones privilegiadas que *no giran*: la matriz solo las estira o encoge. Esas direcciones son los **autovectores** (o vectores propios) y el factor de estiramiento es el **autovalor** λ .

$$A \cdot v = \lambda \cdot v \quad (v \neq 0)$$

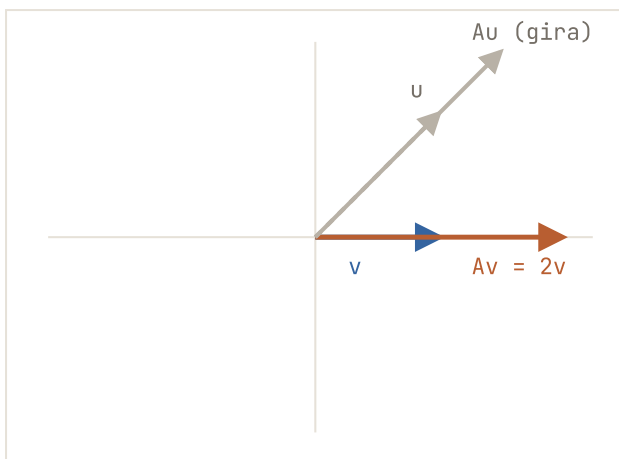


Fig. 8 – v es autovector: solo se alarga ($\lambda=2$).
El vector u cambia de dirección: no lo es.

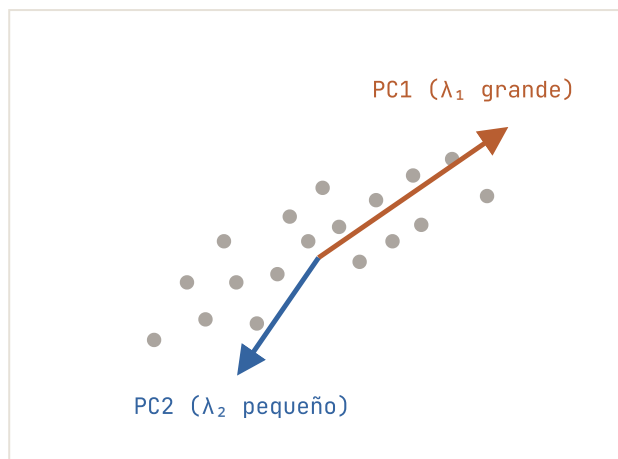


Fig. 9 – PCA: los autovectores de la covarianza son las direcciones de máxima variación de la nube de datos.

11.1 Cómo se calculan a mano

1) Ecuación característica: $\det(A - \lambda I) = 0$

$$A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

$$\det(\begin{bmatrix} 3-\lambda & 1 \\ 1 & 3-\lambda \end{bmatrix}) = (3-\lambda)^2 - 1 = \lambda^2 - 6\lambda + 8 = 0$$

$$\rightarrow \lambda_1 = 4, \lambda_2 = 2$$

2) Para cada λ , resolver $(A - \lambda I)v = 0$:

$$\lambda=4 \rightarrow \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} v = 0 \rightarrow v_1 = (1, 1)/\sqrt{2}$$

$$\lambda=2 \rightarrow \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} v = 0 \rightarrow v_2 = (1, -1)/\sqrt{2}$$

$$\text{Comprobaciones: traza} = 3+3 = 6 = \lambda_1 + \lambda_2 \quad \cdot \quad \det = 9-1 = 8 = \lambda_1 \cdot \lambda_2$$

$$v_1 \cdot v_2 = 1 \cdot 1 + 1 \cdot (-1) = 0 \rightarrow \text{ortogonales, como predice el teorema espectral}$$

TEOREMA ESPECTRAL

Si A es una matriz **simétrica real** ($A = A^T$), entonces: (i) todos sus autovalores son **números reales**; (ii) sus autovectores se pueden elegir **ortonormales** (perpendiculares y de longitud 1); y (iii) A se descompone como

$$A = Q \cdot \Lambda \cdot Q^T \text{ con } Q^T Q = I, \Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$$

Léalo como una receta de tres pasos: Q^T gira los datos hasta alinearlos con las direcciones propias, Λ estira cada eje por su λ , y Q deshace el giro. Toda transformación simétrica, por complicada que parezca, es «rotar, escalar, desrotar». Esa es la garantía teórica que hace que PCA funcione siempre: la matriz de covarianza es simétrica, así que *seguro* tiene una base ortonormal de autovectores reales.

11.2 Consecuencias directas en IA

PCA en cuatro pasos

Centrar los datos $\rightarrow$ calcular $\Sigma = X^T X / (m-1)$
 $\rightarrow$ diagonalizar $\Sigma \rightarrow$ conservar los k
autovectores de mayor λ . La *varianza explicada* es $\lambda_i / \Sigma \lambda$. Así se pasa de 784 dimensiones a 50 perdiendo poco.

SVD: el hermano general

Toda matriz (no solo las simétricas ni cuadradas) admite $A = U \Sigma V^T$. Es la herramienta de compresión, aproximación de rango bajo y recomendadores.

Curvatura y optimización

Los autovalores del hessiano dicen si un punto crítico es mínimo (todos > 0) o punto de silla (signos mixtos). Un cociente $\lambda_{\text{máx}} / \lambda_{\text{mín}}$ alto (mal condicionamiento) explica por qué el descenso de gradiente zigzaguea.

Grafos y estabilidad

Los autovectores del laplaciano de un grafo detectan comunidades (*clustering* espectral) y son la base de las redes neuronales de grafos. El λ dominante también explica los gradientes que explotan o se desvanecen en redes recurrentes.

12 • Matrices dispersas (sparse)

Una matriz es **dispersa** cuando la gran mayoría de sus elementos son cero. No es una curiosidad: es la situación *normal* en los datos reales. Un texto usa unas 20 palabras distintas de un vocabulario de 50 000; un usuario de una plataforma de vídeo ha visto 40

de 100 000 títulos; una red social tiene millones de personas y cada una conoce a unos cientos. La **densidad** se define como (número de valores no nulos) / (total de elementos).

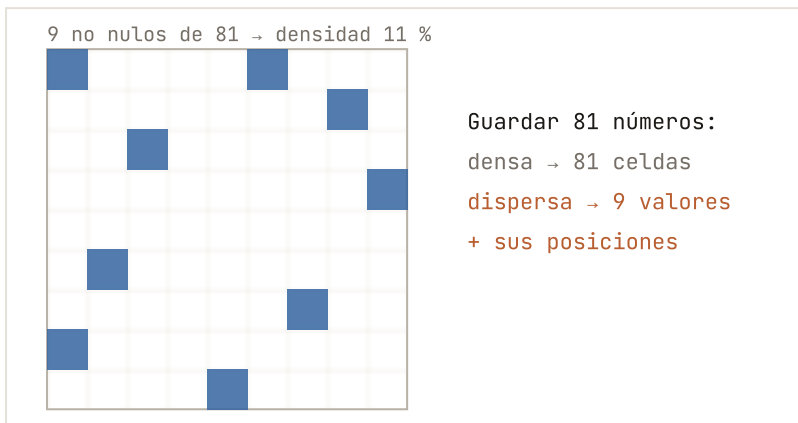


Fig. 10 – En una matriz dispersa solo se almacenan los valores no nulos y sus coordenadas. Con densidades del 0.1 % el ahorro es de tres órdenes de magnitud.

12.1 Formatos de almacenamiento

FORMATO	QUÉ GUARDA	BUENO PARA
COO (coordenadas)	Tres listas: fila, columna, valor	Construir la matriz, leer de un archivo
CSR (por filas)	valores, índices de columna, punteros de fila	Multiplicar matriz por vector; el estándar de facto
CSC (por columnas)	Igual, pero orientado a columnas	Extraer columnas, factorizaciones
Diccionario / hash	(fila, columna) -> valor	Modificar elementos sueltos con frecuencia

```
Matriz [[5, 0, 0],  
        [0, 0, 3],  
        [0, 8, 0]]
```

```
COO: filas=[0,1,2] columnas=[0,2,1] valores=[5,3,8]
```

```
CSR: valores=[5,3,8] col_idx=[0,2,1] ptr_fila=[0,1,2,3]
```

ptr_fila[i] indica dónde empiezan los datos de la fila i en el array de valores.

12.2 Por qué es decisivo en IA

Texto: bolsa de palabras y TF-IDF

Un millón de documentos $\times$ 50 000 palabras en formato denso ocuparía 200 GB; con densidad del 0.04 % cabe en menos de 1 GB. Sin representación dispersa, la minería de texto clásica sería imposible.

Grafos y redes

La matriz de adyacencia de una red social es dispersa y simétrica si el grafo es no dirigido. Las GNN propagan información con productos dispersos matriz-vector.

Recomendadores

La matriz usuarios $\times$ ítems tiene más del 99 % de huecos. El reto es doble: almacenarla dispersa y *completarla* con factorización de rango bajo.

Poda de redes y atención dispersa

Se puede poner a cero el 80–90 % de los pesos de un modelo entrenado sin apenas perder precisión (*pruning*). Los transformers de contexto largo usan patrones de atención dispersos para bajar de coste cuadrático a casi lineal.

ADVERTENCIA PRÁCTICA

Lo disperso no siempre gana: por debajo de un 10–20 % de densidad las estructuras dispersas ahorran memoria y tiempo, pero por encima el coste de guardar los índices y los accesos irregulares a memoria las hace *más lentas* que una matriz densa, que aprovecha mejor la caché y las unidades vectoriales de la GPU. Además, muchas operaciones densifican el resultado sin avisar: al invertir una matriz dispersa, la inversa suele ser densa.

13 • Todo junto en código (NumPy / SciPy)

Cada concepto del curso tiene una línea de código que lo ejecuta. Conviene practicar comprobando a mano los resultados que devuelve la máquina.

```

# representación
import numpy as np
X = np.random.randn(64, 784) # 64 ejemplos, 784 características

# operaciones con vectores
u, v = X[0], X[1]
np.dot(u, v); np.linalg.norm(u) # producto punto y norma L2
cos = u @ v / (np.linalg.norm(u) * np.linalg.norm(v))

# transformación lineal = capa de una red
W = np.random.randn(784, 128) * 0.01
H = X @ W # (64, 128)

# rango, núcleo, determinante
np.linalg.matrix_rank(W) # rango numérico
A = np.array([[3., 1.], [1., 3.]])
np.linalg.det(A) # 8.0 (vía LU,  $O(n^3)$ )
np.linalg.slogdet(A) # log|det| estable numéricamente

# teorema espectral (matriz simétrica → eigh, no eig)
lam, Q = np.linalg.eigh(A) # lam=[2,4], Q ortonormal
np.allclose(Q @ np.diag(lam) @ Q.T, A) # True

# PCA en tres líneas
Xc = X - X.mean(0)
S = Xc.T @ Xc / (len(Xc) - 1)
lam, Q = np.linalg.eigh(S); Z = Xc @ Q[:, ::-1][:, :50]

# matrices dispersas
from scipy.sparse import csr_matrix
S = csr_matrix([[5, 3, 8], ([0, 1, 2], [0, 2, 1])), shape=(3, 3))
S.nnz / (S.shape[0] * S.shape[1]) # densidad = 0.33

```

14 • Mapa conceptual y glosario

Los conceptos del curso encajan en una sola cadena de razonamiento: **vector** → **combinación lineal** → **independencia** → **base y dimensión** → **transformación lineal** → **matriz** → **rango, núcleo e imagen** → **determinante** → **autovalores** → **teorema espectral**

→ **aplicaciones.** Si en algún punto se pierde el hilo, vuelva al eslabón anterior: cada uno solo necesita el precedente.

CONCEPTO	EN UNA FRASE	USO EN IA
Vector	Lista ordenada de números; un punto o flecha	Un dato, un embedding
Producto punto	Medida de alineación entre dos vectores	Similitud, atención, neurona
Independencia lineal	Ninguno es combinación de los otros	Detectar características redundantes
Espacio vectorial	Conjunto cerrado bajo suma y escalado	El «lugar» donde viven los datos
Base / dimensión	Sistema de coordenadas mínimo	Espacio latente, número de factores
Transformación lineal	Función que respeta suma y escalado	Capa de una red neuronal
Rango	Dimensiones de información que sobreviven	Compresión, LoRA, cuellos de botella
Núcleo	Lo que la matriz manda al cero	Información perdida, soluciones múltiples
Determinante	Factor de cambio de volumen; $0 \Rightarrow$ singular	Gaussianas, flujos normalizantes
Matriz simétrica	$A = A^T$, espejo en la diagonal	Covarianza, kernels, grafos
Autovalor / autovector	Dirección que no gira y su factor de escala	PCA, curvatura, clustering espectral
Teorema espectral	Toda simétrica es «rotar, escalar, desrotar»	Garantía teórica de PCA
Matriz dispersa	Casi todo son ceros; se guardan solo los no nulos	Texto, grafos, recomendadores, poda

15 • Ejercicios resueltos

Intente resolverlos con papel antes de mirar la solución. Un concepto de álgebra lineal no se entiende hasta que se calcula al menos una vez a mano.

EJERCICIO 1 • VECTORES

Dados $u = (1, 2, 2)$ y $v = (3, 0, 4)$: calcule $u \cdot v$, $\|u\|$, $\|v\|$ y la similitud del coseno.

¿Son ortogonales?

$u \cdot v = 3+0+8 = 11$ • $\|u\| = \sqrt{9} = 3$ • $\|v\| = \sqrt{25} = 5$ • $\cos \theta = 11/15 \approx 0.733 \rightarrow \theta \approx 42.8^\circ$. No son ortogonales (el producto punto no es cero).

EJERCICIO 2 • INDEPENDENCIA LINEAL

¿Son independientes $(1,2)$, $(2,4)$ y $(0,1)$? ¿Cuál es la dimensión de su span?

No: $(2,4) = 2 \cdot (1,2)$, luego hay dependencia. Además, tres vectores en $\mathbb{R}^2$ nunca pueden ser independientes (no puede haber más vectores independientes que dimensiones). El span es todo $\mathbb{R}^2$, dimensión 2, porque $(1,2)$ y $(0,1)$ ya son independientes entre sí.

EJERCICIO 3 • RANGO Y NÚCLEO

Para $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$: halle $\text{rank}(A)$, $\det(A)$, una descripción del núcleo y diga si es invertible.

$\det = 1 \cdot 4 - 2 \cdot 2 = 0 \rightarrow$ singular, no invertible. $F_2 - 2 \cdot F_1$ deja $\begin{bmatrix} 1 & 2 \\ 0 & 0 \end{bmatrix}$:
 $\text{rank} = 1$. Núcleo: $x + 2y = 0 \rightarrow$ todos los múltiplos de $(2, -1)$; nulidad 1.
Comprobación: $1 + 1 = 2 =$ número de columnas.

EJERCICIO 4 • DETERMINANTE 3×3 POR DOS MÉTODOS

Calcule el determinante de $A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix}$ por Sarrus y por cofactores.

Sarrus: $(1 \cdot 1 \cdot 0 + 2 \cdot 4 \cdot 5 + 3 \cdot 0 \cdot 6) - (3 \cdot 1 \cdot 5 + 2 \cdot 0 \cdot 0 + 1 \cdot 4 \cdot 6) = 40 - 39 = 1$.
Cofactores por la 1ª fila: $1 \cdot (1 \cdot 0 - 4 \cdot 6) - 2 \cdot (0 \cdot 0 - 4 \cdot 5) + 3 \cdot (0 \cdot 6 - 1 \cdot 5) = -24 + 40 - 15 = 1$. ✓ Como $\det \neq 0$, A es invertible y de rango completo.

EJERCICIO 5 • TEOREMA ESPECTRAL

Sea $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, simétrica. Halle sus autovalores y autovectores, compruebe que son ortogonales y escriba $A = Q\Lambda Q^T$.

$$\det(A - \lambda I) = (2 - \lambda)^2 - 1 = 0 \rightarrow \lambda = 3 \text{ y } \lambda = 1.$$

$$\lambda = 3 \rightarrow v_1 = (1, 1)/\sqrt{2} \cdot \lambda = 1 \rightarrow v_2 = (1, -1)/\sqrt{2} \cdot v_1 \cdot v_2 = 0 \checkmark$$

$Q = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{bmatrix}$, $\Lambda = \text{diag}(3, 1)$. Traza 4 = 3+1 $\checkmark$, $\det 3 = 3 \cdot 1 \checkmark$. Al ser ambos autovalores positivos, A es definida positiva.

EJERCICIO 6 • MATRICES DISPERSAS

Un corpus tiene 200 000 documentos y un vocabulario de 40 000 palabras; cada documento usa de media 120 palabras distintas. Calcule la densidad y compare la memoria necesaria en formato denso y disperso (8 bytes por valor, 4 por índice).

$$\text{No nulos} \approx 200\,000 \cdot 120 = 24 \cdot 10^6 \text{ de } 8 \cdot 10^9 \text{ celdas} \rightarrow \text{densidad} = 0.3 \, \%$$

$$\text{Denso: } 8 \cdot 10^9 \cdot 8 \text{ B} = 64 \text{ GB. CSR: } 24 \cdot 10^6 \cdot (8+4) \text{ B} + \text{punteros} \approx 0.29 \text{ GB.}$$

Ahorro $\approx 220\times$. Es la diferencia entre «imposible» y «cabe en un portátil».

Cómo seguir

El siguiente paso natural del programa es la **descomposición en valores singulares (SVD)**, que generaliza el teorema espectral a cualquier matriz, y después el **cálculo multivariante** (gradientes, jacobianos y hessianos), donde el álgebra lineal se convierte en la maquinaria del entrenamiento por descenso de gradiente. Recomendación de estudio: por cada hora de teoría, media hora de cálculo a mano y media hora comprobándolo en código.